JavaScript Interview Notes

Theory + Examples + Outputs + Interview Answers

1. var, let and const

Theory: var, let and const are used to declare variables. var is function-scoped and allows redeclaration and reassignment. let is block-scoped and allows reassignment but not redeclaration. const is block-scoped, must be initialized, and cannot be reassigned.

```
var age = 21;
age = 22;
console.log(age); // 22

let city = "Chennai";
city = "Madurai";
console.log(city); // Madurai

const country = "India";
// country = "USA"; // Error
```

Key points: var → function scope; let → block scope; const → block scope. Prefer const by default and use let when reassignment is needed.

> **Interview Answer:** var is function-scoped and allows redeclaration/reassignment. let is block-scoped and allows reassignment but not redeclaration. const is block-scoped, cannot be reassigned, and must be initialized.

2. Scope

Theory: Scope determines where a variable can be accessed in a program.

Global Scope

```
let name = "Tamil";

function greet() {
    console.log(name); // accessible
}

greet();
console.log(name); // accessible
```

Function Scope

```
function test() {
    let age = 21;
    console.log(age); // OK
}

test();
// console.log(age); // ReferenceError
```

Block Scope

```
if (true) {
    let x = 10;
    const y = 20;
    console.log(x, y); // OK
}

// console.log(x); // Error
// console.log(y); // Error
```

Important: var is function-scoped, while let and const are block-scoped. Inner scopes can access variables from outer scopes, but an outer scope cannot directly access variables declared in an inner scope.

> **Interview Answer:** Scope defines where a variable is accessible. var has function scope, while let and const have block scope.

3. Hoisting and Temporal Dead Zone (TDZ)

Hoisting: JavaScript processes declarations before executing the code. var is hoisted and initialized with undefined. let and const are hoisted but remain uninitialized until execution reaches their declaration.

```
console.log(a);
var a = 10;

// Output: undefined

console.log(b);
let b = 20; // ReferenceError: b is in TDZ
```

TDZ: The Temporal Dead Zone is the period between entering a scope and reaching the let/const declaration. Accessing the variable during this period causes a ReferenceError.

> **Interview Answer:** let and const are hoisted, but they remain uninitialized in the Temporal Dead Zone until their declaration is reached. var is hoisted and initialized with undefined.

4. Closure

Theory: A closure occurs when an inner function remembers and can access variables from its outer lexical scope even after the outer function has finished executing.

```
function createCounter() {
    let count = 0;

    return function () {
        count++;
        console.log(count);
    };
}

const counter = createCounter();

counter(); // 1
counter(); // 2
counter(); // 3
```

The returned function keeps access to count. This is useful for private state, callbacks, function factories, and maintaining state.

> **Interview Answer:** A closure is a function together with access to its surrounding lexical environment, allowing it to remember outer variables even after the outer function has finished.

5. Normal Function vs Arrow Function and this

Normal function: has its own this; for a method call such as user.greet(), this refers to the object that called the method.

```
const user = {
    name: "Tamil",
    greet: function () {
        console.log(this.name);
    }
};

user.greet(); // Tamil
```

Arrow function: does not have its own this. It lexically inherits this from the surrounding scope.

```
const user = {
    name: "Tamil",
    greet: () => {
        console.log(this.name);
    }
};

user.greet(); // does not use user as this
```

Useful case: arrow callbacks can preserve the surrounding this.

```
const user = {
    name: "Tamil",
    greet: function () {
        setTimeout(() => {
            console.log(this.name);
        }, 1000);
    }
};

user.greet(); // Tamil
```

Arrow functions also do not have their own arguments object and cannot be used as constructors with new.

> **Interview Answer:** A normal function has its own this, whose value depends on how it is called. An arrow function does not have its own this; it inherits this lexically from the surrounding scope.

6. Spread and Rest Operators

Same syntax (...), different purpose. Spread expands/unpacks values. Rest collects remaining values.

Spread

```
const a = [1, 2, 3];
const b = [...a, 4, 5];

console.log(b); // [1, 2, 3, 4, 5]

const user = { name: "Tamil", age: 21 };
const updated = { ...user, age: 22 };
```

Rest

```
function add(...numbers) {
    return numbers.reduce((sum, n) => sum + n, 0);
}

console.log(add(10, 20, 30)); // 60
```

Rest must be the last parameter in a function parameter list.

> **Interview Answer:** Spread expands or unpacks arrays/objects, while rest collects multiple values into an array or object pattern.

7. call(), apply(), and bind()

These methods are used mainly to control the value of this for normal functions.

```
const user = { name: "Tamil" };

function greet(city, country) {
    console.log(this.name, city, country);
}

greet.call(user, "Chennai", "India");
greet.apply(user, ["Chennai", "India"]);
```

```
const boundGreet = greet.bind(user);
boundGreet("Chennai", "India");
```

Difference: call executes immediately with individual arguments. apply executes immediately with an array of arguments. bind returns a new function and does not execute it immediately.

Important: call/apply/bind do not change an arrow function's lexical this.

> **Interview Answer:** call and apply invoke immediately; call receives arguments individually and apply receives an array. bind returns a new function with this bound for later execution.

8. Promise

Theory: A Promise represents the eventual result of an asynchronous operation. Its states are pending, fulfilled, and rejected.

```
const promise = new Promise((resolve, reject) => {
    const success = true;

    if (success) {
        resolve("Data received");
    } else {
        reject("Something went wrong");
    }
});

promise
    .then(result => console.log(result))
    .catch(error => console.log(error))
    .finally(() => console.log("Finished"));
```

then handles fulfillment, catch handles rejection, and finally runs after settlement regardless of success or failure.

> **Interview Answer:** A Promise is an object representing the eventual completion or failure of an asynchronous operation. It can be pending, fulfilled, or rejected.

9. Promise Chaining and Promise Static Methods

Promise Chaining

```
Promise.resolve(10)
    .then(value => {
        return value * 2;
    })
    .then(value => {
        return value + 5;
    })
    .then(value => {
        console.log(value); // 25
    });
```

The value or Promise returned by one then becomes the input to the next then.

Static Methods

```
Promise.all([p1, p2, p3]);       // all must fulfill
Promise.allSettled([p1, p2, p3]); // waits for all
Promise.race([p1, p2]);          // first settled wins
Promise.any([p1, p2]);            // first fulfilled wins
Promise.resolve(value);           // fulfilled Promise
Promise.reject(error);            // rejected Promise
```

Memory trick: all = everyone succeeds; allSettled = everyone finishes; race = first settled; any = first success.

> **Interview Answer:** Promise chaining connects multiple then handlers, with returned results passed to the next step. Static methods coordinate or create Promises, including all, allSettled, race, any, resolve, and reject.

10. async / await

Theory: async makes a function return a Promise. await waits for a Promise's result inside an async function and makes Promise-based code easier to read.

```
async function getUsers() {
    try {
        const response = await fetch("/api/users");
        const data = await response.json();

        console.log(data);
    } catch (error) {
        console.log(error);
    }
}

getUsers();
```

await pauses the current async function; it does not block the entire JavaScript runtime.

> **Interview Answer:** An async function always returns a Promise. await waits for a Promise's result inside the async function. try/catch is commonly used for error handling.

11. Event Loop

Theory: JavaScript executes synchronous code on the call stack. The runtime handles asynchronous operations such as timers and network activity. When callbacks become ready, queues hold them until the event loop can schedule them for JavaScript execution.

```
console.log("A");

setTimeout(() => {
    console.log("B");
}, 0);

Promise.resolve().then(() => {
    console.log("C");
});

console.log("D");

// Output:
// A
// D
// C
// B
```

After synchronous code finishes, Promise callbacks are microtasks and are processed before the next task such as the timer callback.

> **Interview Answer:** The event loop coordinates the call stack and asynchronous queues. After current synchronous code finishes, microtasks are processed before the next task.

12. Microtasks vs Macrotasks (Tasks)

Microtasks: Promise then/catch/finally callbacks and queueMicrotask. **Tasks:** commonly timer callbacks such as setTimeout/setInterval, plus other runtime task sources.

```
console.log("1");

setTimeout(() => console.log("2"), 0);

Promise.resolve().then(() => console.log("3"));

console.log("4");
```

```
// Output:
// 1
// 4
// 3
// 2
```

The current synchronous code finishes first. Then the microtask queue is drained before the next task is processed.

> **Interview Answer:** Microtasks have priority over the next task. Promise callbacks are microtasks, while timer callbacks are tasks. Microtasks are processed after current synchronous code and before the next task.

13. Shallow Copy vs Deep Copy

Shallow copy: creates a new top-level object, but nested objects/arrays still share references.

```
const user = {
    name: "Tamil",
    address: { city: "Chennai" }
};

const copy = { ...user };

copy.address.city = "Madurai";

console.log(user.address.city); // Madurai
```

Deep copy: nested objects/arrays are copied independently.

```
const user = {
    name: "Tamil",
    address: { city: "Chennai" }
};

const copy = structuredClone(user);

copy.address.city = "Madurai";

console.log(user.address.city); // Chennai
```

JSON.parse(JSON.stringify(obj)) can deep-copy simple JSON-like data but has limitations. structuredClone is generally preferable when its supported data types fit the use case.

> **Interview Answer:** Shallow copy copies the top level but shares nested references. Deep copy creates independent nested structures too.

14. Debouncing, Throttling, and Currying

Debouncing

Debouncing delays execution until a specified period passes without another trigger. Common use: search boxes and API calls while typing.

```
function debounce(fn, delay) {
    let timer;

    return function (...args) {
        clearTimeout(timer);

        timer = setTimeout(() => {
            fn(...args);
        }, delay);
    };
}

const search = debounce(value => {
    console.log("Searching:", value);
}, 500);
```

Throttling

Throttling limits how often a function can execute during a continuous stream of events. Common use: scroll, resize, and mouse movement.

```
function throttle(fn, delay) {
    let lastTime = 0;

    return function (...args) {
        const now = Date.now();

        if (now - lastTime >= delay) {
            fn(...args);
            lastTime = now;
        }
    };
}
```

Currying

Currying transforms a multi-argument function into a sequence of functions that each receive one argument.

```
const add = (a) => (b) => (c) => a + b + c;

console.log(add(10)(20)(30)); // 60
```

Interview Answer: Debouncing waits until triggering stops. Throttling limits execution frequency. Currying transforms a function into a sequence of one-argument functions.

15. Event Bubbling and Event Delegation

Event Bubbling

When an event occurs on a child element, it can propagate upward through its ancestors.

```
const parent = document.getElementById("parent");
const child = document.getElementById("child");

child.addEventListener("click", () => {
    console.log("Button clicked");
});

parent.addEventListener("click", () => {
    console.log("Parent clicked");
});

// Clicking the button:
// Button clicked
// Parent clicked
```

event.target is the actual element where the event started. event.currentTarget is the element whose listener is currently executing.

```
parent.addEventListener("click", event => {
    console.log(event.target);
    console.log(event.currentTarget);
});
```

event.stopPropagation() prevents the event from continuing through propagation.

Event Delegation

Event delegation uses bubbling by putting one listener on a parent and determining which child triggered the event.

```
const list = document.querySelector("ul");

list.addEventListener("click", event => {
    if (event.target.matches("li")) {
        console.log(event.target.textContent);
```

```
    }
});
```

Delegation is useful for many child elements and dynamically added elements because one parent listener can handle them.

> **Interview Answer:** Bubbling is the event propagation mechanism from child to parent. Event delegation is a technique that uses bubbling to handle many child events with one parent listener.

16. Callback Function

Theory: A callback is a function passed as an argument to another function, which can execute it when needed. A callback may be synchronous or asynchronous.

```
function greet(name) {
    console.log("Hello " + name);
}

function processUser(callback) {
    callback("Tamil");
}

processUser(greet);
// Hello Tamil
```

Asynchronous callback

```
console.log("Start");

setTimeout(() => {
    console.log("Later");
}, 1000);

console.log("End");

// Start
// End
// Later
```

Callbacks are also used by array methods such as map and forEach. Deeply nested asynchronous callbacks can lead to callback hell; Promises and async/await provide cleaner alternatives.

> **Interview Answer:** A callback is a function passed to another function to be executed later or when a particular operation occurs. Callbacks can be synchronous or asynchronous.

17. setTimeout() vs setInterval()

setTimeout: executes a callback once after a delay. **setInterval:** schedules a callback repeatedly at an interval.

```
setTimeout(() => {
    console.log("Runs once");
}, 2000);

const id = setInterval(() => {
    console.log("Runs repeatedly");
}, 1000);

// Stop the interval:
clearInterval(id);

// Cancel a timeout before it runs:
// clearTimeout(timeoutId);
```

A timer does not guarantee exact execution time. Its callback becomes eligible after the delay, then waits for the event loop and call stack to allow it to execute.

Interview Answer: setTimeout runs once after a delay; setInterval repeatedly schedules a callback. clearTimeout cancels a timeout and clearInterval stops an interval.

18. Final Interview Quick Revision

Topic	Remember
var / let / const	var = function scope; let/const = block scope
Scope	Where a variable can be accessed
Hoisting / TDZ	var → undefined; let/const → TDZ before declaration
Closure	Inner function remembers outer variables
Arrow function	No own this; lexical this
Spread / Rest	Spread expands; Rest collects
call / apply / bind	call now / apply array / bind later
Promise	pending → fulfilled/rejected
Promise chaining	Returned result flows to next then
Promise static	all, allSettled, race, any, resolve, reject
async / await	Cleaner Promise syntax
Event loop	Coordinates stack and async queues
Microtask / Task	Microtasks are processed before next task
Shallow / Deep	Nested references shared vs independent
Debounce / Throttle	Wait for stop vs limit frequency
Currying	One argument at a time
Bubbling / Delegation	Child→parent propagation / one parent listener
Callback	Function passed to another function
Timers	Timeout once; interval repeatedly

Best preparation order: var/let/const → scope → hoisting/TDZ → closures → functions/this → callbacks → promises → async/await → event loop → microtasks/tasks → debounce/throttle → DOM events → remaining advanced topics.